The University of Jordan
Faculty of Engineering and Technology
Department of Computer Engineering

Programming of Networks Protocol
Course Final Project – Fall2025
Eng. Asma Abdel Karim

In this course project, you are required to design and implement a network-based **client/server platform** that manages chatrooms between users using **TCP API** in Java.

In this platform, the server is responsible for managing multiple chatrooms and handling communication between connected clients. Each client that connects to the server can either create a new chatroom, join an existing chatroom, or log off.

When the program starts, the user (client) must connect to the server by providing a nickname. After a successful connection, the user is presented with the following menu:

1. Start a new chatroom
2. Join an open chatroom
3. Log off

Start a New Chatroom

- If the user selects this option, they must provide a passcode for the chatroom.
- Other users must enter this passcode in order to join the chatroom.
- Once created, the chatroom must be visible to other users as an available chatroom to join.

Join an Open Chatroom

- If the user selects this option, the client must display a numbered list of all currently open chatrooms.
- For each chatroom, the list must show:
  - The nickname of the chatroom owner
  - The date and time the chatroom was created
- The user selects the chatroom by entering its corresponding number.
- The user is then prompted to enter the chatroom passcode:
  - If the passcode is correct, the user enters the chatroom and receives a confirmation message.
  - If the passcode is incorrect, the user is notified and returned to the main menu.
- Each time a new user joins a chatroom, all other users in that chatroom must be notified with the message:
  "<user_nickname> has joined!"

Chatroom Behavior

- A conversation can begin in a chatroom once at least one user (in addition to the chatroom owner) has joined.
- Any message sent by a user must be broadcast to all other users in the same chatroom.
- All users in a chatroom must be able to send and receive messages concurrently.
- When a message is displayed to users, it must be prefixed with the sender's nickname in the following format:
  <user_nickname>: <message>
- Only the chatroom owner is allowed to close the chatroom.
  - The chatroom is closed when the owner enters the command exit.
  - When the chatroom is closed, all users must be notified and returned to the main menu.

The server must:

- Accept multiple client connections concurrently
- Support multiple open chatrooms in parallel
- Keep track of:
  - Connected players
  - Open and running challenges

Proper handling of concurrent access to shared resources is required to avoid race conditions and data inconsistencies. Use synchronization (e.g., thread-safe data structures or locks) where necessary.

Your final submission must include the following:

1. Source code for both server and client.
2. A README file with instructions on how to run the project.
3. A short report (2-4 pages) describing:
   - System design (UML Class Diagram).
   - Implementation details (briefly describe how your client/server architecture works and provide a brief description of methods in your program).